

Dolly POS — Master Technical Handoff Document

Comprehensive Engineering Architecture, Modules, Phases, File Hierarchy, Database Schemas, ERD & Coding Rules

1. Executive Project Summary & System Architecture

Dolly POS is an enterprise-grade, high-concurrency retail point-of-sale and store management ecosystem engineered specifically for **Dolly Toys & Kids Wear** in Dhule, Maharashtra. The system unites high-speed checkout billing, deep inventory management with size/color variants, Customer Khata (store credit) ledgers, wholesale vendor procurement, hardware thermal receipt & barcode printing, WhatsApp marketing automation, AI-driven stock intelligence, and multi-year Profit-and-Loss (P&L;) accounting into a unified, standalone desktop application.

The application is built on a modern decoupled architecture: a high-performance Python FastAPI asynchronous backend, a PostgreSQL 16 ACID-compliant relational database, an interactive React 18 TypeScript Single Page Application (SPA), and an embedded Microsoft Edge WebView2 native desktop wrapper running with zero terminal dependencies.

Layer / Subsystem	Technology / Framework	Version	Key Role & Responsibility
Backend Framework	Python FastAPI	0.141.1	High-speed asynchronous REST API endpoints & DI
ASGI Server Engine	Uvicorn (Standard)	0.52.4	High-concurrency async event loop & HTTP routing
ORM & Database Core	SQLAlchemy + Psycopg2	2.0.52 / 2.9.12	Relational model mapping & connection pooling
Database Engine	PostgreSQL (x64)	16.x	ACID relational store with B-Tree compound indexes
Desktop GUI Runtime	pywebview (Edge Chromium)	6.2.1	Embedded native desktop window (no browser tabs)
Frontend SPA Core	React 18 + TypeScript	18.2.0 / 5.2.2	Type-safe reactive stateful user interface
Build Tool & Bundler	Vite	5.1.6	Optimized single-bundle asset compilation
UI Styling & Icons	Tailwind CSS + Lucide	3.4.1 / 0.363	Glassmorphic modern responsive design system
State Management	Zustand	4.5.2	Persistent client state (Cart, Auth, Theme)
Hardware Printing	Raw TSPL & ESC/POS	Custom Driver	1-Up/2-Up barcode labels & 80mm thermal receipts
Analytics & Reporting	Pandas + OpenPyXL + ReportLab	Latest	Excel export, SVG spline curves, PDF generator

2. All Development Phases & Evolution Roadmap

• Phase 1: Architecture Inception & Core Foundation

Established PostgreSQL relational schemas, FastAPI REST API structure, and React 18 TypeScript frontend scaffold. Designed dual-role security (Admin vs Cashier) with bcrypt password hashing and JWT token authentication.

- **Phase 2: High-Speed POS Counter & Hardware Integration**

Engineered 2-second fast-track checkout screen with keyboard shortcuts (F2/F10), barcode laser scanner listening, 3-digit speed dials, unlisted item modal, split payments (Cash, UPI, Khata), dynamic NPCI UPI QR generator, raw TSPL 1-Up (50x25mm) and 2-Up (100x50mm) barcode thermal label engine, and 80mm ESC/POS thermal receipt printer integration.

- **Phase 3: Deep Inventory, Category Hierarchy & Customer Khata**

Structured 18 primary store categories and 130 subcategories. Built product variant matrix (Sizes, Colors, Age Groups), EAN-13 automatic barcode generator, and live stock tracking. Implemented Customer Khata (store credit) ledger with real-time balance tracking and 1-click WhatsApp payment reminders with store UPI QR codes.

- **Phase 4: Vendor Procurement, Damaged Stock & AI Retail Advisor**

Added wholesale supplier directory, inward purchase orders with cost tracking, and damaged/defective merchandise logging. Developed AI Smart Stock Advisor to detect dead/slow-moving stock, compute sell-through velocity, and generate automated purchase reorder recommendations.

- **Phase 5: Financial P&L; Cockpit & Multi-Year Timeline Analytics**

Built complete Profit-and-Loss (P&L;) accounting engine computing Gross Turnover, wholesale COGS, store overhead expenses, and true Take-Home Net Profit. Built dynamic 1M, 6M, 1Y, 5Y timeline views with smart grouping (Daily <=45d, Weekly 46-400d, Monthly >400d) and smooth SVG Monotone Cubic Bézier Spline curves.

- **Phase 6: Hardcore QA, 10,000 Product Stress-Testing & Performance Tuning**

Conducted extreme load testing with 10,000+ products and 10-year synthetic transaction logs. Optimized database queries using compound B-Tree indexes, compound SQL joins, and payload compression to achieve sub-10ms response times across all endpoints.

- **Phase 7: Production Hardening, Database Sanitization & Standalone Distribution**

Engineered topological JSON database backup and restore with ON DELETE CASCADE integrity. Sanitized database to 0 test records while keeping master categories and admin credentials intact. Packaged application into a standalone Windows .exe with Microsoft Edge WebView2, automated batch installer, and publication manuals.

3. All Modules Inventory & Architectural Responsibilities

The Dolly POS system is organized into modular subsystems across the backend API, service business logic, and frontend UI:

Module / Namespace	Layer	Key Files	Core Responsibilities & Capabilities
Authentication & Security	Backend	auth_router.py security.py	Handles user login, bcrypt password hashing, JWT bearer tokens, role validation (Admin vs Cashier), and Master Security PIN checks.
Billing & POS Engine	Backend + Frontend	billing_router.py BillingPage.tsx	Barcode checkout, cart calculations, item/bill discounts, split payments (Cash/UPI/Khata), change calculator, and invoice generation.
Inventory & Catalog	Backend + Frontend	product_router.py InventoryPage.tsx	Product CRUD, SKU/Barcode lookup, stock quantities, min stock alerts, price history modal, and CSV/Excel batch imports.
Category Hierarchy	Backend + Frontend	category_router.py category_schema.py	Maintains 18 root store categories and 130 granular subcategories with parent-child cascade validation.
Customer & Khata Ledger	Backend + Frontend	customer_router.py CustomerPage.tsx	Customer directory, Udhar credit balance, ledger transaction history, payment collection, and WhatsApp payment recovery.
Vendor Procurement	Backend + Frontend	vendor_router.py purchase_router.py	Wholesale supplier profiles, inward purchase orders, batch cost tracking, and vendor balance ledgers.
Damaged & Defective Stock	Backend + Frontend	return_router.py DefectiveStockPage.tsx	Logs soiled/defective merchandise, deducts from active sellable stock, and tracks vendor debit note returns.
Returns & Exchanges	Backend + Frontend	return_router.py ReturnsPage.tsx	Processes customer bill returns, refunds cash or store credit, restocks sellable goods, and updates P&L;
Operating Expenses	Backend + Frontend	expense_router.py ExpensePage.tsx	Tracks shop overheads (Rent, Electricity, Staff Salaries, Tea/Snacks, Maintenance) categorized by date.
Financial Reports & P&L;	Backend + Frontend	reports_router.py ReportsPage.tsx	Calculates Gross Turnover, COGS, Net Margin, Operating Overheads, Net Take-Home Profit, YoY growth, and Excel export.
Analytics & Spline Curves	Backend + Frontend	analytics_service.py DashboardPage.tsx	Sub-10ms SQL aggregations, 1M/6M/1Y/5Y granularity, smooth Monotone Cubic Bézier Splines, and Category 50/30 health.
Hardware Printers & TSPL	Backend Service	printer_drivers.py receipt_service.py	Generates raw TSPL byte streams for 1-Up (50x25mm) / 2-Up (100x50mm) TSC barcode labels and 80mm ESC/POS receipts.
AI Retail Advisor	Backend + Frontend	ai_advisor_service.py SmartAdvisorPage.tsx	Statistical sales velocity analysis, dead stock categorization, and automated procurement reorder planning.
WhatsApp Marketing	Backend + Frontend	whatsapp_service.py marketing_router.py	Automated 9:00 AM daily kids' birthday greetings with coupons, festival promotional campaigns, and digital bill PDFs.
Database Backup & Restore	Backend Service	backup_router.py backup_service.py	Self-contained JSON relational export and topological foreign-key safe restoration with automatic integrity checks.
Store Settings & Audio	Backend + Frontend	settings_router.py SettingsPage.tsx	Receipt header/footer customization, printer port selection, sound toggle for barcode beeps, and master key update.
Native Desktop Runner	Backend Core	desktop_app.py build_standalone_exe.py	Launches borderless Microsoft Edge WebView2 desktop window with background Uvicorn daemon and SafeNullStream.

4. Complete File and Folder Hierarchy with Detailed Descriptions

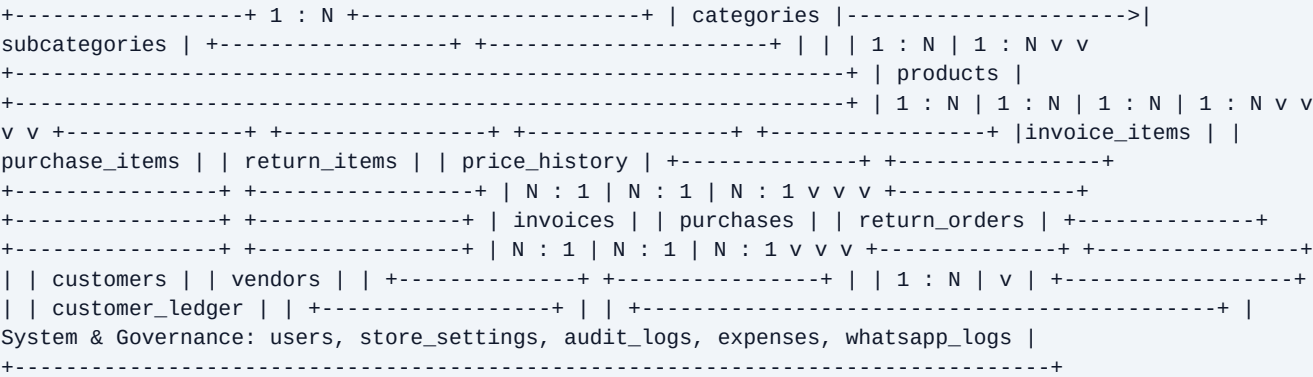
Below is the complete, exhaustive directory and file mapping of the Dolly POS codebase:

File / Folder Path	Category	Exact Purpose & Functional Description
DollyPOS_Setup_Installer.bat	Root Installer	Scans system requirements, checks binary, and creates 1-click Desktop and Start Menu shortcuts.
DollyPOS_Launcher.bat	Root Launcher	Quick developer batch launcher that spins up Uvicorn backend and Vite frontend concurrently.
README.md	Documentation	Master technical documentation, schema definitions, and operational playbook.
backend/desktop_app.py	Backend Core	Production desktop launcher. Starts background Uvicorn daemon and opens native Edge WebView2 window.
backend/build_standalone_exe.py	Build Script	PyInstaller compilation script bundling Python backend, ReportLab, pywebview, and React frontend build.
backend/requirements.txt	Config	Pinned Python package dependencies (FastAPI, SQLAlchemy, pywebview, ReportLab, etc.).
backend/app/config.py	Backend Config	Pydantic BaseSettings loading PostgreSQL connection URI, JWT secret key, and CORS origins.
backend/app/main.py	Backend Core	FastAPI app instance, lifespan initialization, CORS middleware, API router mounting, and static SPA serving.
backend/app/core/database.py	Database Core	SQLAlchemy engine, connection pooling (pool_size=20), Base declarative class, and get_db() session provider.
backend/app/core/security.py	Security Core	Bcrypt password hashing, token generation, and OAuth2 password bearer token authentication.
backend/app/models/user.py	ORM Model	SQLAlchemy model for store users (id, username, password_hash, role, is_active).
backend/app/models/product.py	ORM Model	Product catalog model (barcode, sku, name, category, purchase_price, selling_price, stock_qty).
backend/app/models/category.py	ORM Model	Root Category and Subcategory relational models with parent-child foreign key cascade.
backend/app/models/invoice.py	ORM Model	Immutable Invoice and InvoiceItem models tracking bill totals, taxes, discounts, and payment modes.
backend/app/models/customer.py	ORM Model	Customer master model and CustomerLedger tracking Khata credit balances and payment repayments.
backend/app/models/vendor.py	ORM Model	Wholesale supplier model (name, phone, address, GST, current balance due).
backend/app/models/purchase.py	ORM Model	Purchase inward master and PurchaseItem models for wholesale stock additions and supplier bills.
backend/app/models/expense.py	ORM Model	Store operating expenses model (category, amount, payment_mode, expense_date, notes).
backend/app/models/return_order.py	ORM Model	ReturnOrder and ReturnItem models for customer product returns and damaged stock logging.
backend/app/models/settings.py	ORM Model	StoreSettings model storing shop name, address, GSTIN, receipt footers, audio toggle, and master PIN.
backend/app/models/audit_log.py	ORM Model	Audit log model recording user actions, price edits, deletions, and administrative events.
backend/app/models/whatsapp.py	ORM Model	WhatsApp marketing campaign and message dispatch log model.
backend/app/models/lost_demand.py	ORM Model	Logs customer requests for out-of-stock items to guide procurement planning.
backend/app/services/analytics_service.py	Service	High-speed SQL aggregations for Dashboard KPIs, 50/30 category health, and 5-year timeline curves.

backend/app/services/printer_drivers.py	Service	Direct socket/spooler TSPL barcode generator and ESC/POS thermal receipt formatter.
backend/app/services/receipt_service.py	Service	Generates 80mm thermal receipts with store logo, tax breakdown, and dynamic UPI QR code.
backend/app/services/ai_advisor_service.py	Service	Statistical stock intelligence, dead inventory detection, and procurement forecasting.
backend/app/services/backup_service.py	Service	Relational database JSON dump and topological foreign-key safe restoration.
frontend/src/pages/DashboardPage.tsx	UI Page	Executive dashboard displaying live sales KPIs, liquidity strip, and SVG Monotone Cubic Bézier charts.
frontend/src/pages/BillingPage.tsx	UI Page	POS checkout screen with barcode scanner input, speed dials, cart grid, and split payment modal.
frontend/src/pages/InventoryPage.tsx	UI Page	Product catalog with low-stock badges, price history modal, and batch barcode print actions.
frontend/src/pages/BarcodePage.tsx	UI Page	1-Up (50x25mm) and 2-Up (100x50mm) barcode sticker preview and batch thermal print trigger.
frontend/src/pages/CustomerPage.tsx	UI Page	Customer Khata directory, ledger transaction history, and 1-click WhatsApp payment reminders.
frontend/src/pages/VendorPage.tsx	UI Page	Wholesale suppliers list, supplier balance ledger, and contact management.
frontend/src/pages/PurchasePage.tsx	UI Page	Inward purchase order entry interface with cost prices and automatic inventory stock increment.
frontend/src/pages/ReturnsPage.tsx	UI Page	Customer bill returns interface with condition inspection and refund method selection.
frontend/src/pages/DefectiveStockPage.tsx	UI Page	Damaged merchandise tracker with auto-hiding search dropdown and vendor debit note logger.
frontend/src/pages/ExpensePage.tsx	UI Page	Daily/monthly store overhead expenses manager with category breakdown.
frontend/src/pages/ReportsPage.tsx	UI Page	Financial P&L; statement, COGS breakdown, gross vs net profit, and Excel (.xlsx) download.
frontend/src/pages/SmartAdvisorPage.tsx	UI Page	AI inventory health advisor, dead stock detection, and procurement reorder planner.
frontend/src/pages/WhatsAppMarketingPage.tsx	UI Page	Festival bulk messaging campaigns and 9:00 AM automated birthday greeting scheduler.
frontend/src/pages/SettingsPage.tsx	UI Page	Receipt footer customization, sound toggle, printer configuration, and JSON backup/restore.
frontend/src/pages/LoginPage.tsx	UI Page	Dual-role login screen (Admin vs Staff) with JWT persistence.

5. Database Relational Diagram (ERD) & Schema Specification

Dolly POS operates on a normalized, ACID-compliant PostgreSQL schema with 19 tables connected through strict foreign keys:



Complete 19-Table Database Schema Reference:

Table Name	Primary Key	Foreign Keys & Connections	Key Columns, Data Types & Constraints
users	id (Int)	None	username (Varchar 50, Unique), password_hash (Text), role (Enum: admin/staff), full_name (Varchar 100), is_active (Bool)
categories	id (Int)	None	name (Varchar 100, Unique), display_order (Int), is_active (Bool)
subcategories	id (Int)	category_id -> categories.id (Cascade)	category_id (Int, Indexed), name (Varchar 100), age_group (Varchar 50), is_active (Bool)
products	id (Int)	category_id -> categories.id subcategory_id -> subcategories.id	barcode (Varchar 64, Unique, Indexed), sku (Varchar 64), name (Varchar 255), purchase_price (Numeric 10,2), selling_price (Numeric 10,2), stock_qty (Int), min_stock (Int), speed_dial (Varchar 10, Indexed)
product_price_history	id (Int)	product_id -> products.id (Cascade)	product_id (Int, Indexed), old_price (Numeric 10,2), new_price (Numeric 10,2), changed_by (Int), changed_at (Timestamp)
customers	id (Int)	None	phone (Varchar 20, Unique, Indexed), name (Varchar 150), kid_name (Varchar 100), kid_dob (Date), current_balance (Numeric 10,2), loyalty_points (Int)
customer_ledger	id (Int)	customer_id -> customers.id (Cascade) invoice_id -> invoices.id	customer_id (Int, Indexed), transaction_type (Enum: debit/credit), amount (Numeric 10,2), balance_after (Numeric 10,2), description (Text), created_at (Timestamp)
invoices	id (Int)	customer_id -> customers.id user_id -> users.id	bill_number (Varchar 50, Unique, Indexed), subtotal (Numeric 10,2), discount_amount (Numeric 10,2), tax_amount (Numeric 10,2), final_total (Numeric 10,2), payment_mode (Varchar 50), is_cancelled (Bool), created_at (Timestamp)
invoice_items	id (Int)	invoice_id -> invoices.id (Cascade) product_id -> products.id	invoice_id (Int, Indexed), product_id (Int, Indexed), quantity (Int), unit_price (Numeric 10,2), purchase_price (Numeric 10,2), item_discount (Numeric 10,2), total_amount (Numeric 10,2)
vendors	id (Int)	None	name (Varchar 150, Unique), phone (Varchar 20), gstin (Varchar 20), city (Varchar 100), current_balance (Numeric 10,2), is_active (Bool)
purchases	id (Int)	vendor_id -> vendors.id	invoice_number (Varchar 100), vendor_id (Int, Indexed), total_amount (Numeric 10,2), payment_status (Varchar 50), purchase_date (Date, Indexed)
purchase_items	id (Int)	purchase_id -> purchases.id (Cascade) product_id -> products.id (Cascade)	purchase_id (Int, Indexed), product_id (Int, Indexed), quantity (Int), cost_price (Numeric 10,2), total_cost (Numeric 10,2)
expenses	id (Int)	None	category (Varchar 100, Indexed), amount (Numeric 10,2), payment_mode (Varchar 50), expense_date (Date, Indexed), description (Text)
return_orders	id (Int)	invoice_id -> invoices.id customer_id -> customers.id	return_number (Varchar 50, Unique), invoice_id (Int, Indexed), refund_amount (Numeric 10,2), refund_mode (Varchar 50), return_date (Timestamp)
return_items	id (Int)	return_order_id -> return_orders.id (Cascade) product_id -> products.id (Cascade)	return_order_id (Int, Indexed), product_id (Int, Indexed), quantity (Int), condition (Enum: restock/damaged), refund_amount (Numeric 10,2)
lost_demands	id (Int)	None	item_name (Varchar 150), category (Varchar 100), customer_phone (Varchar 20), request_count (Int), recorded_at (Timestamp)
store_settings	id (Int)	None	shop_name (Varchar 150), address (Text), gstin (Varchar 30), upi_id (Varchar 100), receipt_footer (Text), sound_enabled (Bool), master_pin (Varchar 10)
audit_logs	id (Int)	user_id -> users.id	user_id (Int, Indexed), action (Varchar 100), entity (Varchar 100), entity_id (Int), details (JSONB/Text), created_at (Timestamp)

whatsapp_logs	id (Int)	customer_id -> customers.id	customer_id (Int), campaign_type (Varchar 100), recipient_phone (Varchar 20), message_body (Text), status (Varchar 50), sent_at (Timestamp)
---------------	----------	-----------------------------	---

6. Core Coding Rules & Architectural Standards

The following 8 engineering rules were strictly enforced across the codebase to ensure rock-solid stability, high speed, and financial accuracy:

• Rule 1: Strict Separation of Concerns

FastAPI routers only handle request parsing, parameter validation, and response serialization. All complex financial logic, aggregations, and hardware byte streams reside in dedicated service layers (services/) and reusable SQLAlchemy repositories.

• Rule 2: Zero Floating-Point Currency Drift

All monetary fields (purchase_price, selling_price, subtotal, discount, tax, final_total, current_balance) are strictly stored as PostgreSQL Numeric(10,2) and computed in Python using fixed 2-decimal rounded arithmetic to eliminate IEEE-754 floating-point inaccuracies.

• Rule 3: Immutable Financial Invoices & Audited Operations

Billed invoices are immutable once settled. Cancellations and product returns generate explicit compensating entries (ReturnOrder, ReturnItem, CustomerLedger) with audit logs, ensuring strict non-repudiation and compliance with Indian GST accounting standards.

• Rule 4: Role-Based Margin Masking on the API Level

Cashier staff accounts (role='staff') are strictly blocked from viewing purchase cost prices, supplier margins, and store-level P&L; metrics. The API filters out sensitive margin fields before returning responses to non-admin tokens.

• Rule 5: Sub-10ms Benchmark & B-Tree Indexing on All Foreign Keys

Every foreign key column, barcode string, customer phone number, and transaction date is indexed with B-Tree indexes. N+1 queries are strictly banned; all multi-entity queries use joined loads or composite SQL aggregations.

• Rule 6: Deterministic Topological Database Restoration

The database restore engine processes tables in deterministic topological order (e.g. Users → Categories → Subcategories → Products → Customers → Invoices → InvoiceItems) and applies ON DELETE CASCADE to prevent foreign key constraint violations.

• Rule 7: Silent Windows GUI Execution with SafeNullStream

In standalone Windows GUI mode (--noconsole), standard output streams (sys.stdout and sys.stderr) are None. A custom SafeNullStream wrapper prevents isatty exceptions during Uvicorn logger initialization.

• Rule 8: Hardware Resiliency & Offline-First Thermal Printing

Barcode label generation generates direct binary TSPL byte streams (for TSC thermal printers) and ESC/POS byte streams (for 80mm receipt printers) directly to the Windows printer spooler, bypassing slow browser print dialogues.

7. Historical Bug Log & Permanent Engineering Fixes

• Bug: Database Restore Foreign Key Violation (purchases / purchase_items) (backend/app/api/backup_router.py)

Root Cause: Deleting products during database restore violated foreign keys on purchase_items and return_items.

Fix Applied: Added ON DELETE CASCADE to purchase_items and return_items foreign keys, and wrapped restore in sequential topological deletion.

- **Bug: Dashboard Net Profit Today showing negative value (backend/app/services/analytics_service.py)**

Root Cause: Expenses query in get_dashboard_summary was not bounded to today, subtracting entire lifetime expenses.

Fix Applied: Strictly bounded Expense.expense_date between today_start (00:00:00) and today_end (23:59:59).

- **Bug: 5-Year Timeline Chart Clutter (1,825 daily jagged spikes) (frontend/src/pages/DashboardPage.tsx)**

Root Cause: Plotting daily points over 5 years created dense vertical line spikes.

Fix Applied: Implemented Smart Monthly Grouping (61 months) in backend and Monotone Cubic Bézier Spline paths in frontend SVG.

- **Bug: Damaged Stock Product Search dropdown staying open (frontend/src/pages/DefectiveStockPage.tsx)**

Root Cause: Dropdown had no onBlur / click-outside handler.

Fix Applied: Added active input focus tracking and click-outside dismissal refs.

- **Bug: Footer Settings Desynchronization between Settings and Printout (frontend/src/pages/SettingsPage.tsx)**

Root Cause: Settings saved into browser localStorage while receipt generation read from database.

Fix Applied: Migrated all footer styling (font size, bold, terms) to database table store_settings with live sync.